

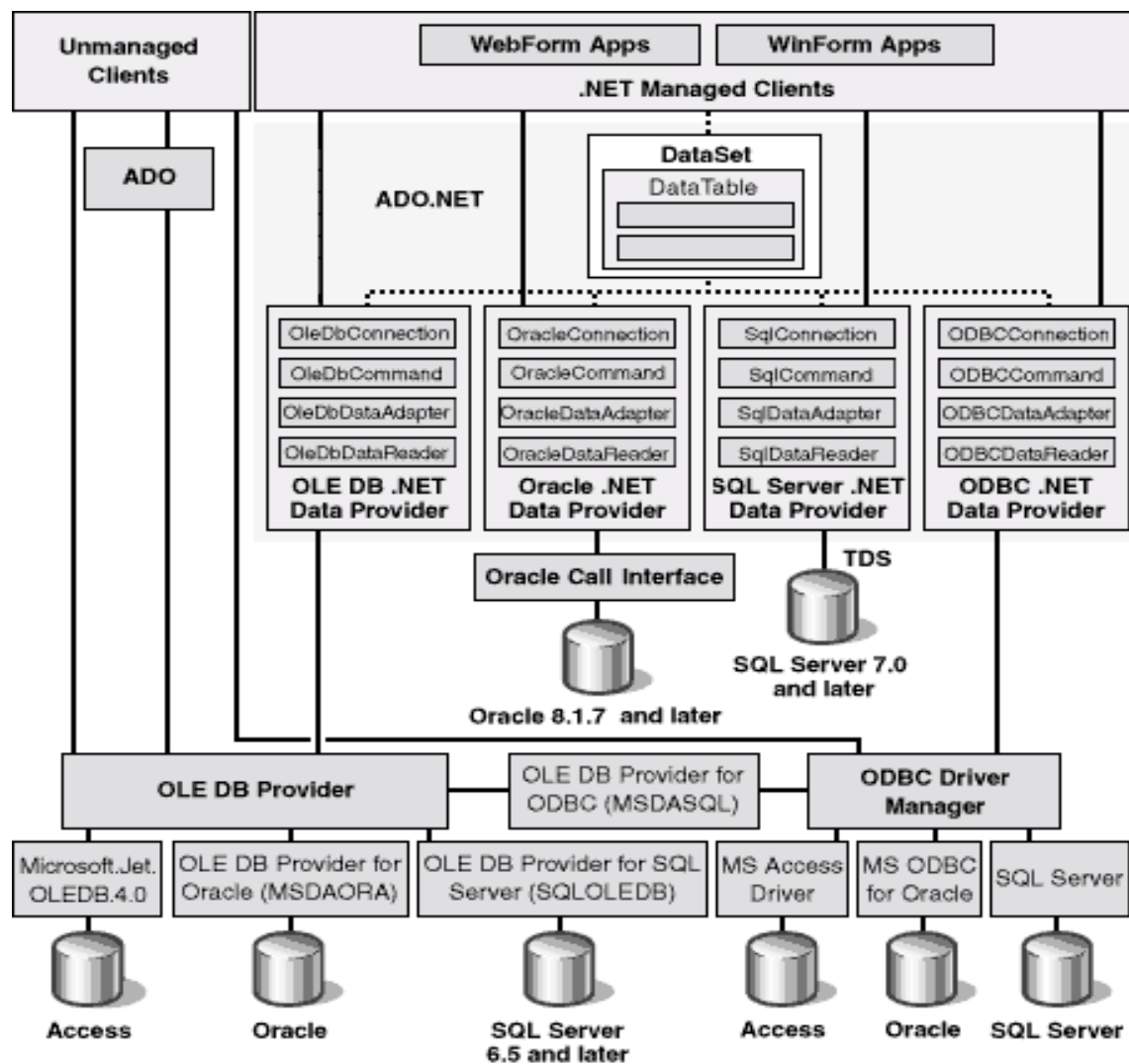
Базы данных

Тема 13

Технология доступа к данным ADO.NET

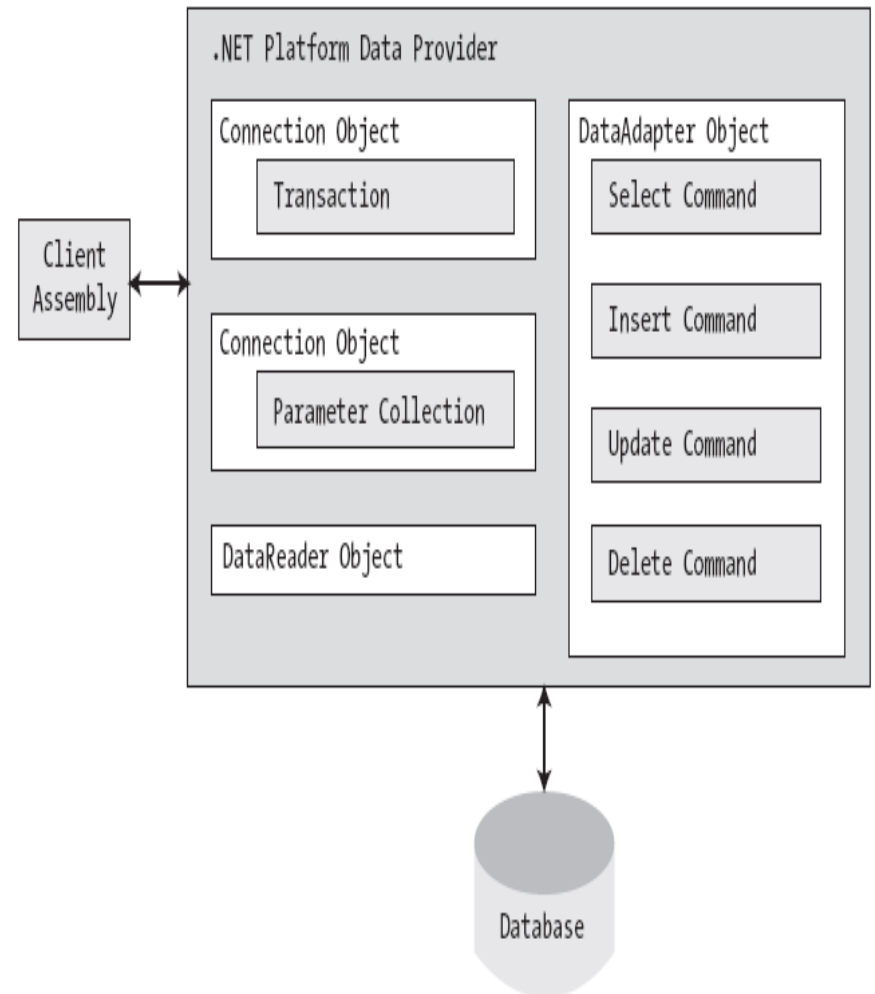
Стек доступа к данным

Доступ к данным
в технологии
ADO.NET
осуществляется
путем
обращения к
специальным
библиотекам -
поставщикам
(провайдерам,
providers),
реализующим
определенный
набор типов
(прежде всего,
классов и
интерфейсов)
доступа к
данным

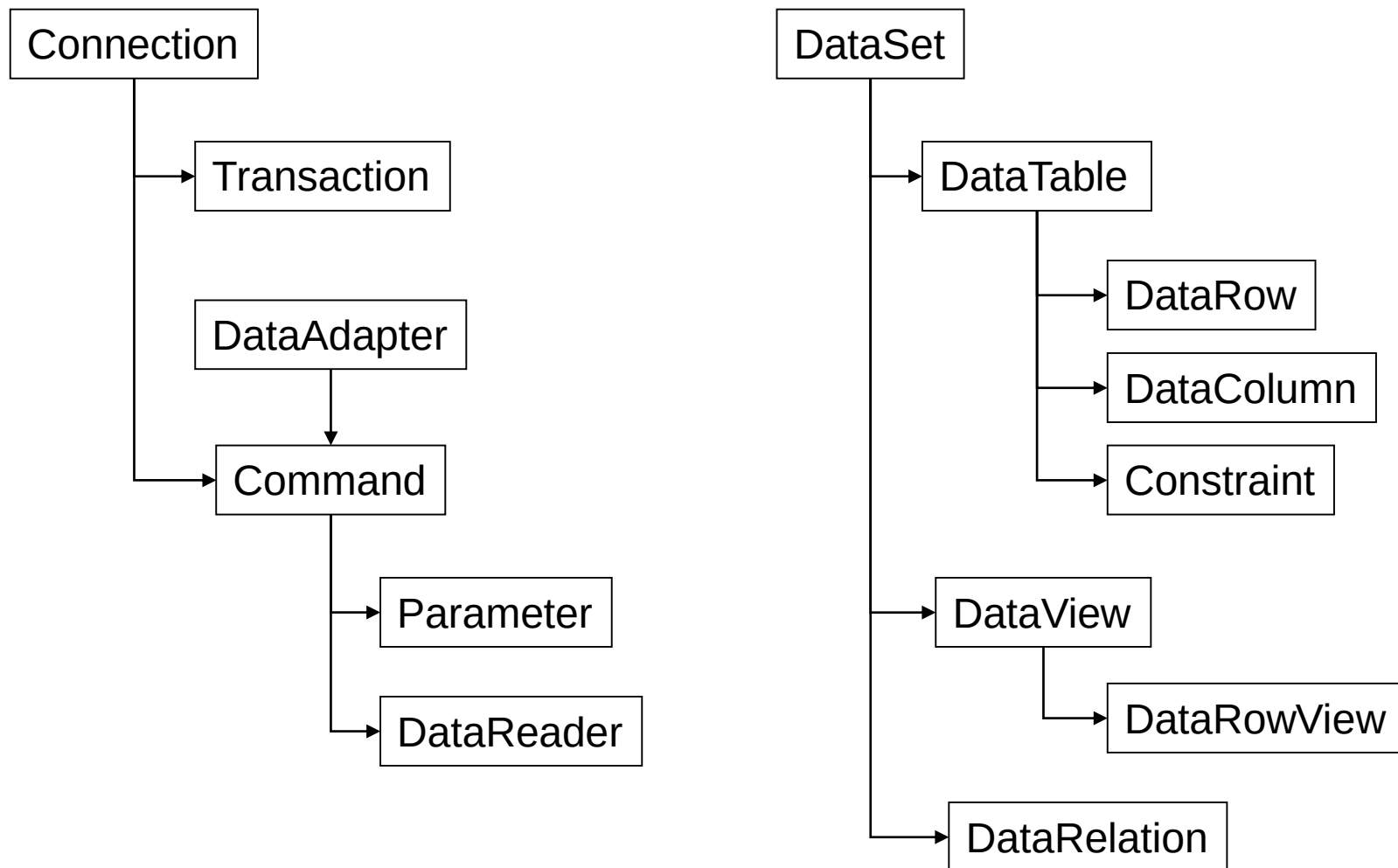


Режимы работы ADO.NET

- режим с поддержанием соединения (связный уровень, *connected layer*):
 - осуществляется непосредственное, явное подключение к источнику данных и отключение от него;
 - используются объекты **Connection**, **Command**, **DataReader**;
- режим без поддержания соединения (несвязный уровень, *disconnected layer*):
 - операции с данными производятся над локальной копией данных источника, представленной объектом **DataSet** и связанными объектами (**DataTable** etc.);
 - используется объект **DataAdapter** для обеспечения взаимодействия приложения и источника данных, при этом операции открытия и закрытия соединения выполняются автоматически по мере необходимости;
 - фактически операции с объектами несвязного уровня можно выполнять вообще без подключения к источнику данных.



Объектная модель ADO.NET



Основные объекты поставщиков данных ADO.NET

Объект	Базовый класс	Реализуемые интерфейсы	Назначение
Connection	DbConnection	IDbConnection	Обеспечивает подключение к источнику данных
Command	DbCommand	IDbCommand	Представляет запрос к источнику данных
DataReader	DbDataReader	IDataReader IDataRecord	Обеспечивает однонаправленный просмотр данных (на основе курсора, открытого на сервере)
DataAdapter	DbDataAdapter	IDataAdapter IDbDataAdapter	Обеспечивает обмен данными между источниками данных и объектами наборов данных (DataSet)
Parameter	DbParameter	IDataParameter IDbDataParameter	Представляет именованный параметр в параметризованном запросе
Transaction	DbTransaction	IDbTransaction	Инкапсулирует транзакцию

Пространство имен System.Data

содержит типы, общие для всех поставщиков данных:

- типы примитивов баз данных (таблицы, ряды, столбцы, ограничения и т.п.);
- интерфейсы, реализуемые поставщиками данных;
- специфические исключения, возникающие при работе с БД (NotNullAllowedException, RowNotInTableException, MissingPrimaryKeyException etc.).

Тип	Назначение
DataSet	Набор данных – локальный кэш данных источника
DataTable	Таблица (в DataSet)
DataRow	Строка (в DataTable)
DataColumn	Столбец (в DataTable)
DataRelation	Отношение (связь) между таблицами
Constraint	Ограничение (для объекта DataColumn)
DataView	Представление, построенное для объекта DataTable
IDataAdapter	Основной интерфейс объекта DataAdapter
IDbDataAdapter	Интерфейс объекта DataAdapter, описывающий дополнительную функциональность
IDataParameter	Интерфейс объекта Parameter
IDataReader	Интерфейс объекта DataReader
IDbCommand	Интерфейс объекта Command
IDbTransaction	Интерфейс объекта Transaction

Интерфейс IDbConnection

```
public interface IDbConnection : IDisposable
{
    string ConnectionString { get; set; }
    int ConnectionTimeout { get; }
    string Database { get; }
    ConnectionState State { get; }

    IDbTransaction BeginTransaction();
    IDbTransaction BeginTransaction(IsolationLevel il);
    void ChangeDatabase(string databaseName);
    void Close();
    IDbCommand CreateCommand();
    void Open();
}
```

Интерфейс IDbTransaction

```
public interface IDbTransaction : IDisposable
{
    IDbConnection Connection { get; }
    IsolationLevel IsolationLevel { get; }

    void Commit();
    void Rollback();
}
```


Интерфейс IDbCommand

```
public interface IDbCommand : IDisposable
{
    string CommandText { get; set; }
    int CommandTimeout { get; set; }
    CommandType CommandType { get; set; }
    IDbConnection Connection { get; set; }
    IDataParameterCollection Parameters { get; }
    IDbTransaction Transaction { get; set; }
    UpdateRowSource UpdatedRowSource { get; set; }

    void Cancel();
    IDataParameter CreateParameter();
    int ExecuteNonQuery();
    IDataReader ExecuteReader();
    IDataReader ExecuteReader(CommandBehavior behavior);
    object ExecuteScalar();
    void Prepare();
}
```

Интерфейсы IDbDataParameter и IDataParameter

```
public interface IDbDataParameter : IDataParameter
{
    byte Precision { get; set; }
    byte Scale { get; set; }
    int Size { get; set; }
}
```

```
public interface IDataParameter
{
    DbType DbType { get; set; }
    ParameterDirection Direction { get; set; }
    bool IsNullable { get; }
    string ParameterName { get; set; }
    string SourceColumn { get; set; }
    DataRowVersion SourceVersion { get; set; }
    object Value { get; set; }
}
```

Интерфейсы IDbDataAdapter и IDataAdapter

```
public interface IDbDataAdapter : IDataAdapter
{
    IDbCommand DeleteCommand { get; set; }
    IDbCommand InsertCommand { get; set; }
    IDbCommand SelectCommand { get; set; }
    IDbCommand UpdateCommand { get; set; }
}
```

```
public interface IDataAdapter
{
    MissingMappingAction MissingMappingAction { get; set; }
    MissingSchemaAction MissingSchemaAction { get; set; }
    ITableMappingCollection TableMappings { get; }

    int Fill(System.Data.DataSet dataSet);
    DataTable[] FillSchema(DataSet dataSet, SchemaType schemaType);
    IDataParameter[] GetFillParameters();
    int Update(DataSet dataSet);
}
```

Интерфейсы IDataReader и IDataRecord

```
public interface IDataReader :  
    IDisposable, IDataRecord  
{  
    int Depth { get; }  
    bool IsClosed { get; }  
    int RecordsAffected { get; }  
  
    void Close();  
    DataTable GetSchemaTable();  
    bool NextResult();  
    bool Read();  
}
```

```
public interface IDataRecord  
{  
    int FieldCount { get; }  
    object this[ string name ] { get; }  
    object this[ int i ] { get; }  
  
    bool GetBoolean(int i);  
    byte GetByte(int i);  
    char GetChar(int i);  
    DateTime GetDateTime(int i);  
    Decimal GetDecimal(int i);  
    float GetFloat(int i);  
    short GetInt16(int i);  
    int GetInt32(int i);  
    long GetInt64(int i);  
    ...  
    bool IsDBNull(int i);  
}
```

Категории провайдеров

- провайдеры непосредственного доступа (native providers) - обеспечивают доступ к данным путем непосредственного обращения к соответствующим источникам данных с использованием их собственных протоколов доступа:
 - **SQL Client .NET Data Provider** (MS .Net Framework, MS SQL Server 7.0 и выше);
 - **Microsoft SQL Server Mobile** (MS .Net Framework, MS SQL Server Mobile Edition);
 - **Oracle Client .NET Data Provider** (MS .Net Framework, Oracle 8.1.7 и выше);
 - **Oracle Data Provider, ODP.NET** (.Net Framework Data Provider for Oracle, Oracle 8.1.7 и выше);
- провайдеры-посредники (bridge providers) - обеспечивают доступ к данным путем использования других (более ранних технологий доступа):
 - **OLE DB .NET Data Provider** (MS .Net Framework, источники данных OLEDB):
 - SQLOLEDB - Microsoft OLE DB Provider for SQL Server;
 - MSDAORA - Microsoft OLE DB Provider for Oracle;
 - Microsoft.Jet.OLEDB.4.0 - OLE DB Provider for Microsoft Jet;
 - **ODBC .NET Data Provider** (MS .Net Framework, источники данных ODBC):
 - SQL Server;
 - Microsoft ODBC for Oracle;
 - Microsoft Access Driver (*.mdb).

<http://www.databasedrivers.com/ado/>

Пространства имен ADO.NET

- пространства имен ADO.NET можно разделить на следующие категории:
 - пространства имен, специфические для конкретного поставщика данных:
 - основное пространство имен (System.Data.SqlClient, Oracle.DataAccess.Client);
 - вспомогательные пространства имен (System.Data.Sql, System.Data.SqlTypes, Oracle.DataAccess.Types);
 - пространства имен, содержащие базовые типы библиотеки ADO.NET:
 - System.Data (определение общих типов, не связанных с конкретным источником данных - DataSet, DataTable, DataRow, IDataAdapter, IDataReader, IDataParameter, IDbTransaction etc.);
 - System.Data.Common (определение абстрактных базовых классов DbXXX).

Именованние объектов доступа к данным и пространства имен

- Connection
- Command
- CommandBuilder
- DataReader
- DataAdapter
- Transaction
- Parameter

Каждый поставщик данных, помимо базовых функций, как правило реализует также некоторые дополнительные функции, специфические для конкретного источника данных (СУБД)

Провайдер	Пространство имен	Префикс
Базовые классы	System.Data.Common (System.Data.dll)	<i>Db...</i>
.NET Framework Data Provider for SQL Server	System.Data.SqlClient (System.Data.dll)	<i>Sql...</i>
.NET Framework Data Provider for SQL Server Mobile	System.Data.SqlServerCe (System.Data.SqlServerCe.dll)	<i>SqlCe...</i>
.NET Framework Data Provider for Oracle	System.Data.OracleClient (System.Data.OracleClient.dll)	<i>Oracle...</i>
Oracle Data Provider, ODP.NET	Oracle.DataAccess.Client (Oracle.DataAccess.dll)	<i>Oracle...</i>
.NET Framework Data Provider for OLE DB	System.Data.OleDb (System.Data.dll)	<i>OleDb...</i>
.NET Framework Data Provider for ODBC	System.Data.Odbc (System.Data.dll)	<i>Odbc...</i>

Доступ к данным: пример (SQL Server)

```
{  
    // Create and open a connection.  
    SqlConnection cn = new SqlConnection();  
    cn.ConnectionString =  
        @"Data Source=(local)\SQLEXPRESS;Integrated Security=SSPI;" +  
        "Initial Catalog=AutoLot";  
    cn.Open();  
  
    // Create a SQL command object.  
    string strSQL = "Select * From Inventory";  
    SqlCommand myCommand = new SqlCommand(strSQL, cn);  
  
    // Obtain a data reader  
    SqlDataReader dr;  
    myDataReader = myCommand.ExecuteReader(CommandBehavior.CloseConnection);  
  
    // Loop over the results.  
    while (dr.Read())  
    {  
        Console.WriteLine("-> Make: {0}, Color: {1}.",  
            dr["Make"].ToString().Trim(), dr["Color"].ToString().Trim());  
    }  
  
    dr.Close();  
}
```


Доступ к данным: пример (Oracle)

```
using System;
using Oracle.DataAccess.Client;
class Sample
{
    static void Main()
    {
        // Connect to Oracle
        string constr = "User Id=scott;Password=tiger;Data Source=oracle";
        OracleConnection con = new OracleConnection(constr);

        con.Open();

        // Display Version Number
        Console.WriteLine("Connected to Oracle " + con.ServerVersion);

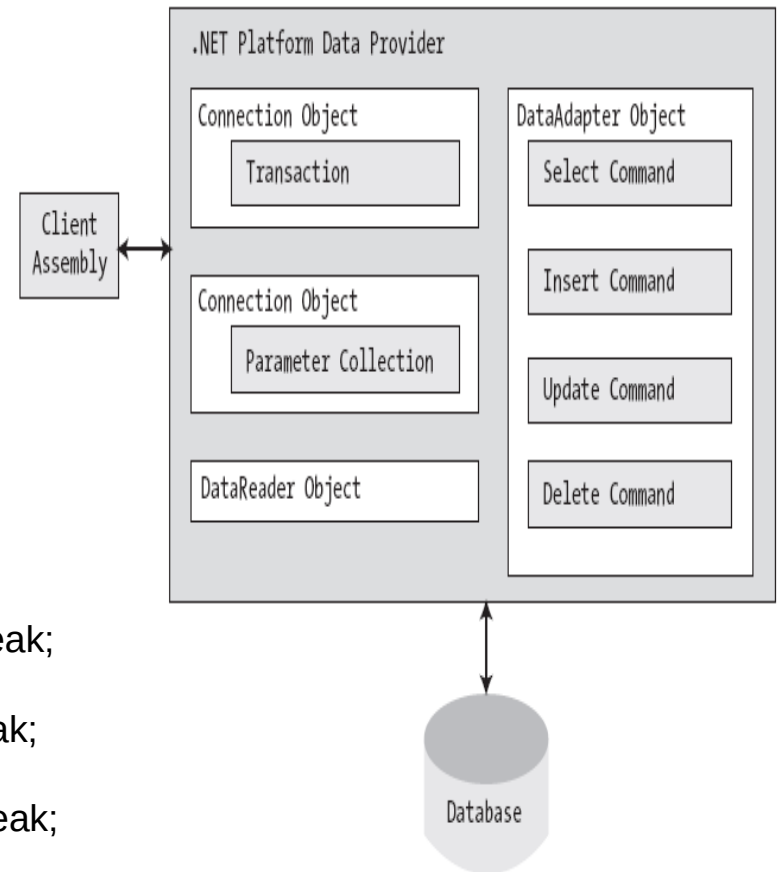
        // Close and Dispose OracleConnection
        con.Close();
        con.Dispose();
    }
}
```

Использование общих базовых интерфейсов

```
public static void OpenConnection(IDbConnection cn)
{
    // Open the incoming connection for the caller.
    cn.Open();
}

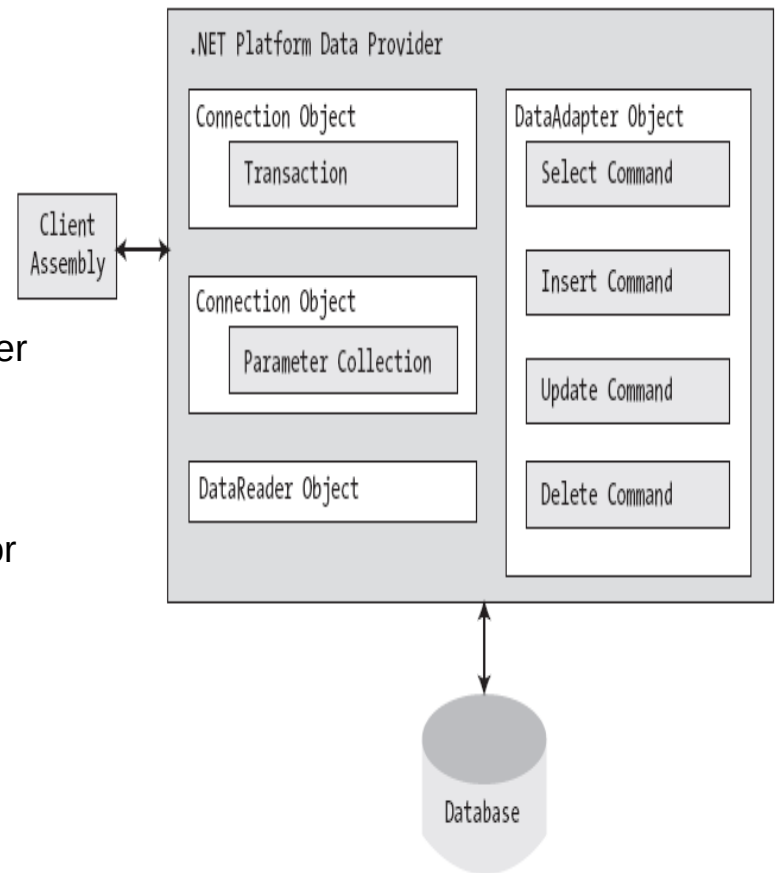
static IDbConnection GetConnection(DataProvider dp)
{
    IDbConnection conn = null;
    switch (dp)
    {
        case DataProvider.SqlServer:
            conn = new SqlConnection();break;
        case DataProvider.OleDb:
            conn = new OleDbConnection();break;
        case DataProvider.Odbc:
            conn = new OdbcConnection();break;
        case DataProvider.Oracle:
            conn = new OracleConnection();break;
    }

    return conn;
}
```



Программирование в ADO.NET 2.0+ : использование общих базовых классов (Oracle)

OracleClientFactory DbProviderFactory
OracleCommand DbCommand
OracleCommandBuilder DbCommandBuilder
OracleConnection DbConnection
OracleConnectionStringBuilder DbConnectionStringBuilder
OracleDataAdapter DbDataAdapter
OracleDataReader DbDataReader
OracleDataSourceEnumerator DbDataSourceEnumerator
OracleException DbException
OracleParameter DbParameter
OracleParameterCollection DbParameterCollection
OracleTransaction DbTransaction



Использование общих базовых классов и функций фабрики объектов поставщика (ADO.NET 2.0): DbProviderFactories.GetFactory() и DbProviderFactory

```
public abstract class DbProviderFactory
```

```
{
```

```
    ...
```

```
    public virtual DbCommand CreateCommand();
```

```
    public virtual DbCommandBuilder CreateCommandBuilder();
```

```
    public virtual DbConnection CreateConnection();
```

```
    public virtual DbConnectionStringBuilder CreateConnectionStringBuilder();
```

```
    public virtual DbDataAdapter CreateDataAdapter();
```

```
    public virtual DbDataSourceEnumerator CreateDataSourceEnumerator();
```

```
    public virtual DbParameter CreateParameter();
```

```
}
```

```
using System.Data.Common;
```

```
// Get the factory for the SQL data provider.
```

```
DbProviderFactory sqlFactory = DbProviderFactories.GetFactory("System.Data.SqlClient");
```

```
...
```

```
// Get the factory for the Oracle data provider.
```

```
DbProviderFactory oracleFactory = DbProviderFactories.GetFactory("System.Data.OracleClient");
```

Использование общих базовых классов и функций фабрики объектов поставщика (ADO.NET 2.0) : пример

```
{
    string constr = "user id=scott;password=tiger;data source=oracle";
    DbProviderFactory factory = DbProviderFactories.GetFactory("Oracle.DataAccess.Client");
    DbConnection conn = factory.CreateConnection();
    try
    {
        conn.ConnectionString = constr;
        conn.Open();
        DbCommand cmd = factory.CreateCommand();
        cmd.Connection = conn;
        cmd.CommandText = "select * from emp";
        DbDataReader reader = cmd.ExecuteReader();
        while (reader.Read()) Console.WriteLine(reader["ENAME"]);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

Регистрация провайдеров ADO.NET 2.0 : файлы конфигурации (machine.config, app.config)

```
<system.data>
  <DbProviderFactories>
    <add name="Odbc Data Provider" invariant="System.Data.Odbc"
description=".Net Framework Data Provider for Odbc" type="System.Data.Odbc.OdbcFactory,
System.Data, Version=2.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"/>
    <add name="OleDb Data Provider" invariant="System.Data.OleDb"
description=".Net Framework Data Provider for OleDb" type="System.Data.OleDb.OleDbFactory,
System.Data, Version=2.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"/>
    <add name="OracleClient Data Provider"
invariant="System.Data.OracleClient" description=".Net Framework Data Provider for Oracle"
type="System.Data.OracleClient.OracleClientFactory, System.Data.OracleClient, Version=2.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089"/>
    <add name="SqlClient Data Provider" invariant="System.Data.SqlClient"
description=".Net Framework Data Provider for SqlServer"
type="System.Data.SqlClient.SqlClientFactory, System.Data, Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089"/>
    <add name="Microsoft SQL Server Compact Data Provider"
invariant="System.Data.SqlServerCe.3.5" description=".NET Framework Data Provider for
Microsoft SQL Server Compact" type="System.Data.SqlServerCe.SqlCeProviderFactory,
System.Data.SqlServerCe, Version=3.5.0.0, Culture=neutral,
PublicKeyToken=89845dcd8080cc91"/>
  </DbProviderFactories>
</system.data>
```

Связный уровень ADO.NET: общий сценарий работы

1. создание, настройка и открытие объекта соединения (Connection);
2. создание и настройка объекта запроса (Command), в том числе:
 - определение объекта соединения для объекта запроса;
 - определение строки запроса;
3. вызов метода ExecuteReader() объекта DataReader для получения результатов запроса;
4. прокрутка результатов выполнения запроса (DataReader.Read()).

```
SqlConnection cn = new SqlConnection();  
  
cn.ConnectionString = @"Data  
Source=(local)\SQLEXPRESS;Integrated Security=SSPI;  
Initial Catalog=AutoLot";  
  
cn.Open();  
  
  
string strSQL = "Select * From Inventory";  
  
SqlCommand myCommand = new SqlCommand(strSQL,  
cn);  
  
  
SqlDataReader myDataReader;  
  
myDataReader = myCommand.ExecuteReader();  
  
  
while (myDataReader.Read())  
{ ...}  
  
myDataReader.Close();  
  
cn.Close()
```

Connection

- содержит необходимую информацию для соединения с сервером;
- содержит информацию о текущем состоянии и позволяет его изменять;
- может использоваться пул соединений.

- BeginTransaction()
- ChangeDatabase()
- ConnectionTimeout
- Database
- DataSource
- GetSchema()
- State

```
using System.Data;
```

```
using System.Data.SqlClient;
```

```
string connectionString = "Data Source=localhost;" +
```

```
"Initial Catalog=Northwind;Integrated Security=SSPI;"
```

```
SqlConnection conn = new SqlConnection(connectionString);
```

```
conn.Open();
```

```
    // работа с базой данных
```

```
conn.Close();
```


Параметры соединения

MS SQL Server

Advanced	
MultipleActiveResultSets	False
Network Library	
Packet Size	8000
Transaction Binding	Implicit Unbind
Type System Version	Latest
Context	
Application Name	.Net SqlClient Data Provider
Workstation ID	
Initialization	
Asynchronous Processing	False
Connect Timeout	15
Current Language	
Pooling	
Enlist	True
Load Balance Timeout	0
Max Pool Size	100
Min Pool Size	0
Pooling	True
Replication	
Replication	False
Security	
Encrypt	False
Integrated Security	True
Password	
Persist Security Info	False
TrustServerCertificate	False
User ID	
Source	
AttachDbFilename	
Context Connection	False
Data Source	.\SQLEXPRESS
Failover Partner	
Initial Catalog	Northwind
User Instance	False

Oracle

Data	
ConnectionString	DATA SOURCE=oracle;USER ID=s
Misc	
Connection Life Time	
Connection Timeout	15
Context Connection	False
Data Source	oracle
DBA Privilege	
Decrement pool size	
Enlist	true
HAEvents	
Increment pool size	
Load Balancing	False
Max Pool Size	100
metadata pooling	True
Min Pool Size	1
Password	*****
Persist Security Info	False
Pooling	True
Proxy Password	
Proxy User	
Statement Cache Purge	False
Statement Cache Size	0
User ID	scott
Validate Connection	False

<http://www.connectionstrings.com/>

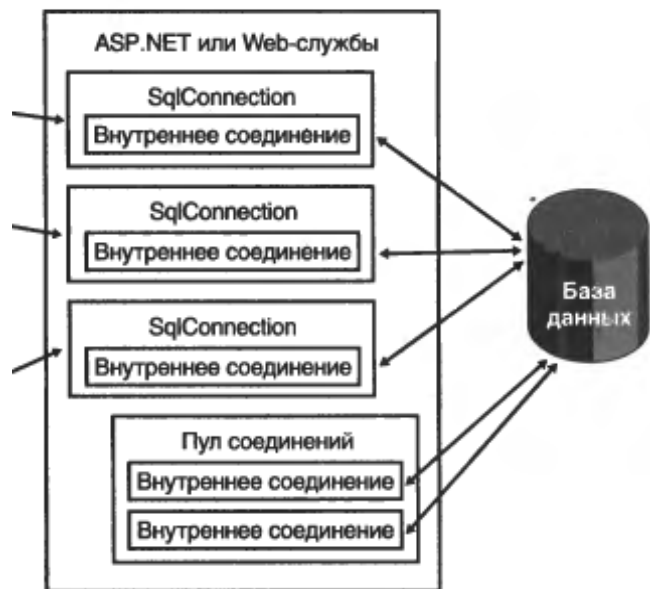
Формирование строки подключения

- непосредственное перечисление необходимых пар имя параметра – значение в строке;
- использование конструктора строк подключения (ConnectionStringBuilder), который обеспечивает строгую типизацию передаваемых параметров для конкретного поставщика данных.

```
SqlConnectionStringBuilder cnStrBuilder = new  
SqlConnectionStringBuilder();  
  
cnStrBuilder.InitialCatalog = "AutoLot";  
  
cnStrBuilder.DataSource = @"(local)\SQLEXPRESS";  
  
cnStrBuilder.ConnectTimeout = 30;  
  
cnStrBuilder.IntegratedSecurity = true;  
  
SqlConnection cn = new SqlConnection();  
cn.ConnectionString = cnStrBuilder.ConnectionString;  
  
....
```

Пул соединений

- пул соединений позволяет улучшить производительность за счет повторного использования установленного физического соединения с источником;
- пул соединений поддерживается поставщиками подключений ADO.NET и по умолчанию используется (*Pooling = true*);



- в момент соединения поставщик определяет:
 - существует ли пул для данной строки подключения;
 - есть ли в данном пуле свободные (неактивные) соединения;
- минимальный размер пула определяется параметром (*MinPoolSize = 0*):
 - в момент открытия первого соединения пула поставщик устанавливает необходимое число физических подключений к источнику;
 - поставщик поддерживает минимальное число соединений в пуле независимо от того, являются ли они активными;
- максимальный размер пула определяется параметрами (*MaxPoolSize = 100*), а в случае, когда достигнут максимальный размер пула и предпринимается попытка открытия нового соединения:
 - производится ожидание освобождения соединения пула в течении *ConnectionTimeout = 15*;
 - по истечении таймута возникает исключение *InvalidOperationException*;
- при закрытии соединения и уничтожении объекта соединения внутреннее соединение с СУБД не разрывается, а помещается в пул для дальнейшего использования;
- соединение в пуле, которое не использовалось, уничтожается по истечении таймута;
- возможна принудительная очистка пулов путем вызова функций *ClearPool()* и *ClearAllPools()*.

Использование файлов конфигурации (app.config) для выбора провайдера и настройки подключения

```
<?xml version="1.0" encoding="utf-8" ?>
<connectionStrings>
  <add name="ASClientCS"
        providerName="System.Data.SqlClient"
        connectionString="Server=192.168.0.92\sqli2005; Database=OneToManyMerge; User ID=sa;
                          Password=Hc6X"/>
</connectionStrings>
```

// Get the factory provider.

```
DbProviderFactory df =
DbProviderFactories.GetFactory(ConfigurationManager.ConnectionStrings["ASClientCS"].ProviderName);
```

// Now make connection object.

```
DbConnection cn = df.CreateConnection();
cn.ConnectionString = ConfigurationManager.ConnectionStrings["ASClientCS"].ConnectionString;
cn.Open();
```

// Make command object.

```
DbCommand cmd = df.CreateCommand();
cmd.Connection = cn;
cmd.CommandText = "Select * From Inventory";
```

```
DbDataReader dr = cmd.ExecuteReader(CommandBehavior.CloseConnection);
while (dr.Read()) {...}
dr.Close();
```

Command

- ПОЗВОЛЯЕТ ВЫПОЛНИТЬ:
 - команду языка SQL (*CommandType == Text*);
 - хранимую процедуру (*CommandType == StoredProcedure*);
- с точки зрения возвращаемых данных запросы могут быть отнесены к одному из трех типов:
 - не возвращающие результатов [INSERT, UPDATE, DELETE] : *ExecuteNonQuery()*;
 - возвращающие одиночное (скалярное) значение [SELECT COUNT(...)...]: *ExecuteScalar()*;
 - возвращающие набор результатов : *ExecuteReader()*;
- SqlCommand позволяет организовать асинхронный доступ к данным:
 - *BeginExecuteReader()* / *EndExecuteReader()*;
 - *BeginExecuteNonQuery()* / *EndExecuteNonQuery()*;
 - *BeginExecuteXmlReader()* / *EndExecuteXmlReader()*;

- *CommandTimeout* (= 30)
- *Connection*
- *Parameters*
- *Cancel()*
- *ExecuteReader()*
- *ExecuteNonQuery()*
- *ExecuteScalar()*
- *ExecuteXmlReader()*
- *Prepare()*

```
public enum
CommandType
{
    StoredProcedure,
    TableDirect,
    Text    // Default value.
}
```

ExecuteNonQuery() : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = conn.CreateCommand();  
cmd.CommandText =  
    "DELETE FROM Customers WHERE CustomerID = 'ALFKI'";  
Console.WriteLine(  
    "RecordsAffected" + cmd.ExecuteNonQuery());  
  
conn.Close();
```

ExecuteScalar() : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = conn.CreateCommand();  
cmd.CommandText = "SELECT COUNT(*) FROM Customers";  
Console.WriteLine(Convert.ToInt32(cmd.ExecuteScalar()));  
  
conn.Close();
```

ExecuteReader() : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = new SqlCommand(  
    "SELECT CustomerID, CompanyName FROM Customers", conn);  
  
SqlDataReader rdr = cmd.ExecuteReader();  
while (rdr.Read()) {  
    Console.WriteLine(  
        rdr["CustomerID"] + " - " + rdr["CompanyName"]);  
}  
rdr.Close();  
conn.Close();
```


DataReader

- обеспечивает последовательный доступ к результатам запроса;
- требует наличия постоянного активного подключения к базе данных в процессе работы;
- не позволяет возвращаться к предыдущим записям;
- не позволяет вносить изменения в данные;
- позволяет прочитать несколько наборов результатов;
- перегруженный индексатор позволяет осуществлять доступ к полям записи как по индексу, так и по имени.

DataReader : пример

```
string connectionString = "Data Source=localhost;" +  
"Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
conn.Open();  
  
SqlCommand cmd = new SqlCommand(  
    "SELECT CustomerID, CompanyName FROM Customers;" +  
    "SELECT OrderID, CustomerID FROM Orders", conn);  
  
SqlDataReader rdr = cmd.ExecuteReader();  
// SqlDataReader rdr = cmd.ExecuteReader(CommandBehavior.CloseConnection);  
  
do {  
    while (rdr.Read()) {  
        Console.WriteLine(rdr[0] + " - " + rdr[1]);  
    }  
    Console.WriteLine();  
} while (rdr.NextResult());  
  
rdr.Close();  
  
conn.Close();
```

Использование параметризованных запросов: Parameter

```
string sql = string.Format("Insert Into Inventory" +  
"(CarID, Make, Color) Values (@CarID, @Make, @Color)");
```

// This command will have internal parameters.

```
using(SqlCommand cmd = new SqlCommand(sql, this.sqlCn))  
{
```

// Fill params collection.

```
SqlParameter param = new SqlParameter();  
param.ParameterName = "@CarID";  
param.Value = id;  
param.SqlDbType = SqlDbType.Int;  
cmd.Parameters.Add(param);
```

```
param = new SqlParameter();  
param.ParameterName = "@Make";  
param.Value = make;  
param.SqlDbType = SqlDbType.Char;  
param.Size = 10;  
cmd.Parameters.Add(param);
```

```
param = new SqlParameter();  
param.ParameterName = "@Color";  
param.Value = color;  
param.SqlDbType = SqlDbType.Char;  
param.Size = 10;  
cmd.Parameters.Add(param);
```

```
cmd.ExecuteNonQuery();
```

```
}
```

- DbType
- Direction
- IsNullable
- ParameterName
- Size
- Value

Выполнение хранимых процедур

```
using (SqlCommand cmd = new SqlCommand("GetPetName", this.sqlCn))
{
    cmd.CommandType = CommandType.StoredProcedure;

    SqlParameter param = new SqlParameter();
    param.ParameterName = "@carID";
    param.SqlDbType = SqlDbType.Int;
    param.Value = carID;
param.Direction = ParameterDirection.Input;
    cmd.Parameters.Add(param);

    param = new SqlParameter();
    param.ParameterName = "@petName";
    param.SqlDbType = SqlDbType.Char;
    param.Size = 10;
param.Direction = ParameterDirection.Output;
    cmd.Parameters.Add(param);

    cmd.ExecuteNonQuery();

    Name = (string)cmd.Parameters["@petName"].Value;
}
```

```
CREATE PROCEDURE GetPetName
    @carID int,
    @petName char(10)
        output
AS
SELECT @petName = PetName
from Inventory
where CarID = @carID
```

Transaction

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();
    SqlCommand command = connection.CreateCommand();
    SqlTransaction transaction;

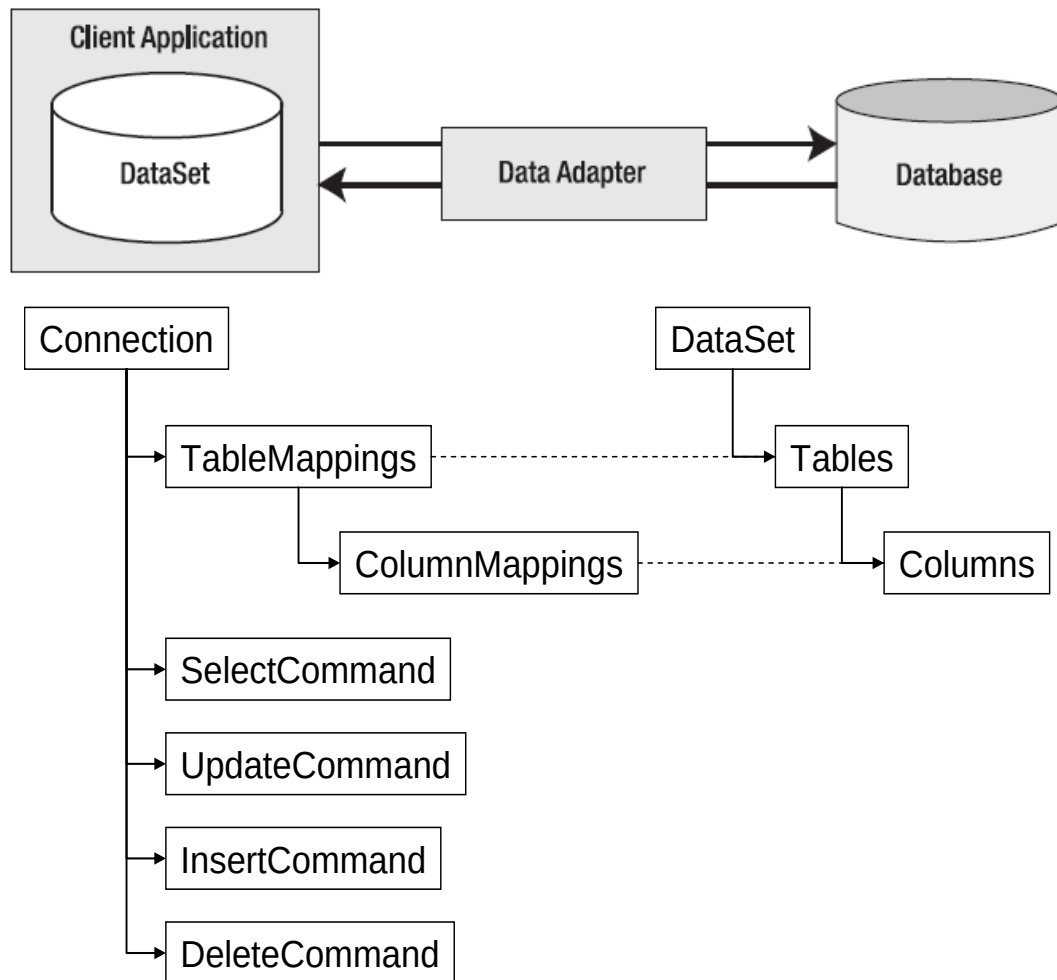
    transaction = connection.BeginTransaction("SampleTransaction");
    command.Connection = connection;
    command.Transaction = transaction;

    try
    {
        command.CommandText = "Insert into Reg (RegID, RegDesc) VALUES (1, 'D1')";
        command.ExecuteNonQuery();
        command.CommandText = "Insert into Reg (RegID, RegDesc) VALUES (2, 'D2')";
        command.ExecuteNonQuery();

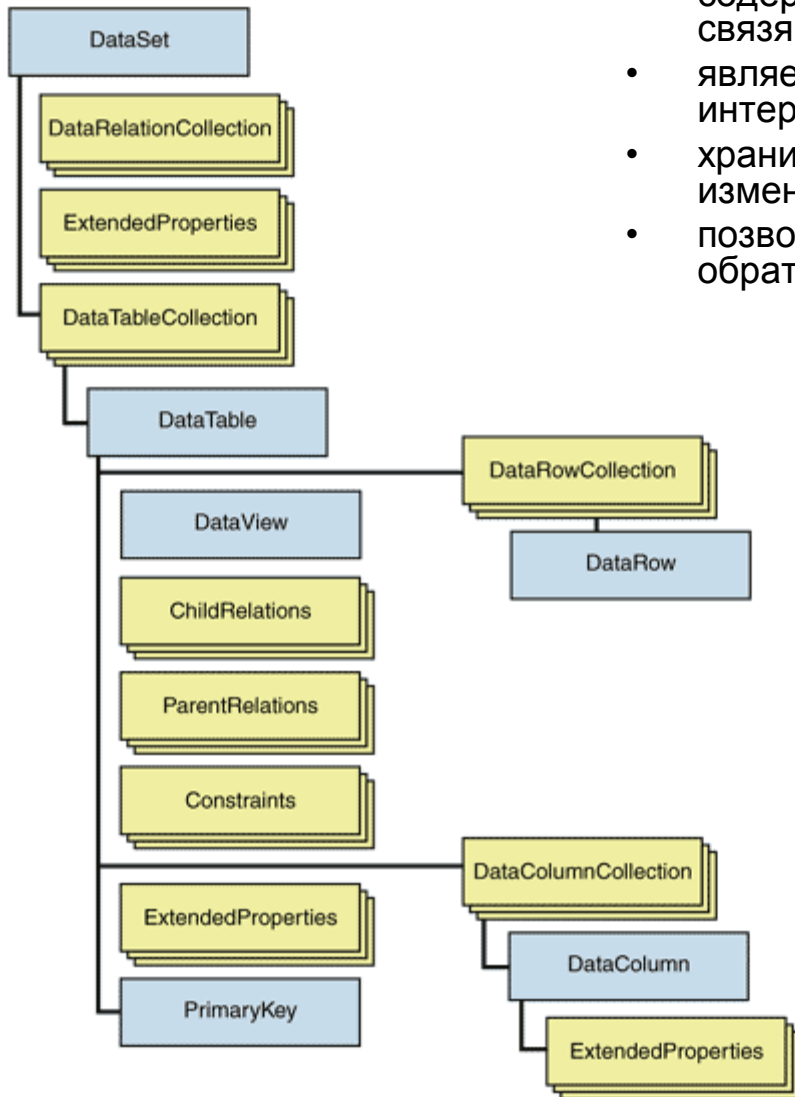
        transaction.Commit();
    }
    catch (Exception ex)
    {
        Console.WriteLine(" Message: {0}", ex.Message);
        try
        {
            transaction.Rollback();
        }
        catch (Exception ex2)
        {
            Console.WriteLine("Rollback Exception Msg: {0}", ex2.Message);
        }
    }
}
```

Несвязный уровень ADO.NET: DataAdapter

- автономно управляет подключением к источнику данных;
- получает результаты запроса и записывает их в DataSet;
- переносит в БД изменения, внесенные в DataSet;
- устанавливает соответствие между результатами запроса и таблицами / столбцами в DataSet.



DataSet



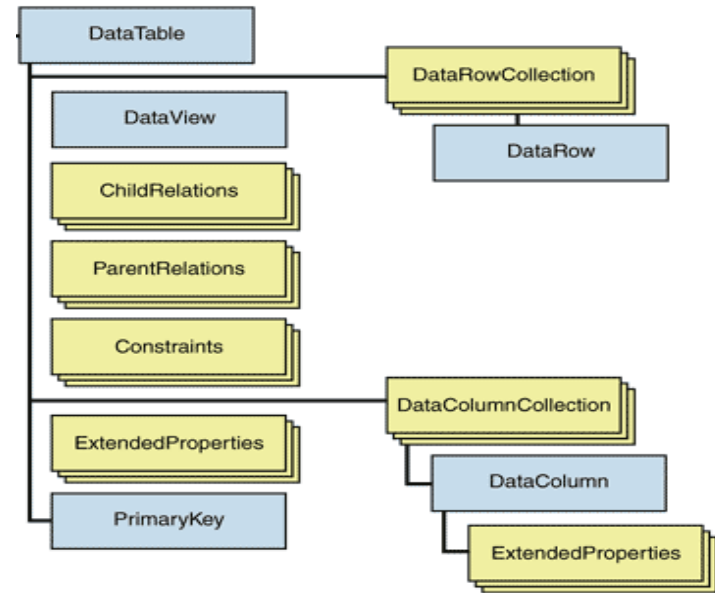
- содержит данные в виде набора таблиц с возможными связями между ними;
- является источником данных для элементов управления интерфейса пользователя;
- хранит локальную копию исходных данных и внесенных изменений;
- позволяет преобразовывать данные в XML документ и обратно, а также в двоичный формат;

- CaseSensitive
- DataSetName
- EnforceConstraints
- HasErrors
- RemotingFormat
- AcceptChanges() / RejectChanges()
- Clear()
- Clone() / Copy()
- GetChanges()
- GetChildRelations()
- GetParentRelations()
- HasChanges()
- Merge()
- ReadXml() / ReadXmlSchema()
- WriteXml() / WriteXmlSchema()

DataTable

- содержит данные в виде набора записей, состоящих из одинаковых наборов полей;
- каждая запись хранится как в исходном варианте, так и в текущем – с учетом внесенных изменений.

- CaseSensitive
- ChildRelations / ParentRelations
- Constraints / PrimaryKey
- Copy()
- DataSet
- DefaultView
- MinimumCapacity
- RemotingFormat
- Select()
- TableName
- ReadXml() / ReadXmlSchema()
- WriteXml() / WriteXmlSchema()



```
DataSet ds = new DataSet();
DataTable tbl = ds.Tables.Add("Customers");

DataColumn col = tbl.Columns.Add("CustomerID");
col.AllowDBNull = false;
col.MaxLength = 5;
col.Unique = true;

DataColumn col = tbl.Columns.Add("CompanyName");
col.MaxLength = 40;
```


DataColumn

- определяет столбец в таблице и его свойства: тип, значение по умолчанию, может ли содержать NULL, может ли значение быть изменено;
- задает выражение для вычисляемых столбцов.

- AllowDBNull (*true*) / Unique
- AutoIncrement / AutoIncrementSeed / AutoIncrementStep
- Caption
- ColumnMapping
- ColumnName (*Column1, Column2, ...*).
- DataType
- DefaultValue
- Expression
- Ordinal
- ReadOnly (*false*)
- Table

```
DataColumn carIDColumn = new DataColumn("CarID", typeof(int));  
carIDColumn.Caption = "Car ID";  
carIDColumn.ReadOnly = true;  
carIDColumn.AllowDBNull = false;  
carIDColumn.Unique = true;  
  
DataColumn carMakeColumn = new DataColumn("Make", typeof(string));  
DataColumn carColorColumn = new DataColumn("Color", typeof(string));
```

DataRow

- соответствует одной записи в таблице;
- может быть в одном из состояний:
 - `Unchanged`;
 - `Detached`;
 - `Added`;
 - `Modified`;
 - `Deleted`;
- содержит следующие версии значений полей:
 - `Original`;
 - `Current`;
 - `Proposed`;
 - `Default`.

- `HasErrors` / `GetColumnsInError()` / `GetColumnError()` / `ClearErrors()` / `RowError`
- `ItemArray`
- `RowState`
- `Table`
- `AcceptChanges()` / `RejectChanges()`
- `BeginEdit()` / `EndEdit()` / `CancelEdit()`
- `Delete()`
- `IsNull()`

```
DataTable inventoryTable = new
    DataTable("Inventory");

inventoryTable.Columns.AddRange(new
    DataColumn[]
    { carIDColumn, carMakeColumn,
      carColorColumn,
    carPetNameColumn });
```

```
carRow = inventoryTable.NewRow();
carRow[0] = 0;
carRow[1] = "Saab";
carRow[2] = "Red";
carRow[3] = "Sea Breeze";
inventoryTable.Rows.Add(carRow);
```

Состояния строк объекта DataRow

```
// temp DataTable for testing.  
DataTable temp = new DataTable("Temp");  
temp.Columns.Add(new DataColumn("TempColumn", typeof(int)));
```

// RowState = Detached.

```
DataRow row = temp.NewRow();
```

// RowState = Added.

```
temp.Rows.Add(row);
```

// RowState = Added.

```
row["TempColumn"] = 10;
```

// RowState = Unchanged.

```
temp.AcceptChanges();
```

// RowState = Modified.

```
row["TempColumn"] = 11;
```

// RowState = Deleted.

```
temp.Rows[0].Delete();
```

Объекты несвязного уровня : пример

```
DataSet carsInventoryDS = new DataSet("Car Inventory");

carsInventoryDS.ExtendedProperties["TimeStamp"] = DateTime.Now;
carsInventoryDS.ExtendedProperties["DataSetID"] = Guid.NewGuid();
carsInventoryDS.ExtendedProperties["Company"] = "Intertech";

DataColumn carIDColumn = new DataColumn("CarID", typeof(int));
carIDColumn.Caption = "Car ID";
carIDColumn.ReadOnly = true;
carIDColumn.AllowDBNull = false;
carIDColumn.Unique = true;
DataColumn carMakeColumn = new DataColumn("Make",
    typeof(string));
DataColumn carColorColumn = new DataColumn("Color",
    typeof(string));

DataTable inventoryTable = new DataTable("Inventory");
inventoryTable.Columns.AddRange(new DataColumn[]
    { carIDColumn, carMakeColumn, carColorColumn, carPetNameColumn
    });

carsInventoryDS.Tables.Add(inventoryTable);
```

DataRelation

- устанавливает связь один-к одному, один-ко-многим или многие-ко-многим между двумя столбцами или наборами столбцов;
- по умолчанию, для родительских столбцов автоматически добавляется ограничение *UniqueConstraint*, а для дочерних – *ForeignKeyConstraint*;
- позволяет определять родительские и дочерние записи для указанной записи:
 - **DataRow.GetParentRow();**
 - **DataRow.GetParentRows();**
 - **DataRow.GetChildRows().**

```
public DataRelation ( string relationName,  
                    DataColumn parentColumn,  
                    DataColumn childColumn,  
                    bool createConstraints )      // = true
```

```
DataRelation dr = new DataRelation("CustomerOrder",  
autoLotDS.Tables["Customers"].Columns["CustID"],           // parent table  
autoLotDS.Tables["Orders"].Columns["CustID"]);             // child table  
autoLotDS.Relations.Add(dr);
```

Constraint

- *Primary Key* – указывает столбцы, входящие в первичный ключ;
- *Unique* – указывает столбцы, значения которых должно быть уникально в таблице;
- *Foreign Key* – указывает столбцы, входящие во внешний ключ, а также задает правила поведения ограничений ссылочной целостности (*Cascade*, *SetNull*, *SetDefault*, *None*).

```
ForeignKeyConstraint custOrderFK = new ForeignKeyConstraint("CustOrderFK",  
    custDS.Tables["CustTable"].Columns["CustomerID"],  
    custDS.Tables["OrdersTable"].Columns["CustomerID"]);  
  
custOrderFK.DeleteRule = Rule.None;  
  
custDS.Tables["OrdersTable"].Constraints.Add(custOrderFK);
```

```
DataTable custTable = custDS.Tables["Customers"];  
  
UniqueConstraint custUnique = new UniqueConstraint(new DataColumn[]  
    {custTable.Columns["CustomerID"], custTable.Columns["CompanyName"]});  
  
custDS.Tables["Customers"].Constraints.Add(custUnique);
```

DataTableReader

- является аналогом DataReader для таблиц несвязного уровня (DataTable)

```
static void PrintTable(DataTable dt)
{
    // Get the DataTableReader type.
    DataTableReader dtReader = dt.CreateDataReader();

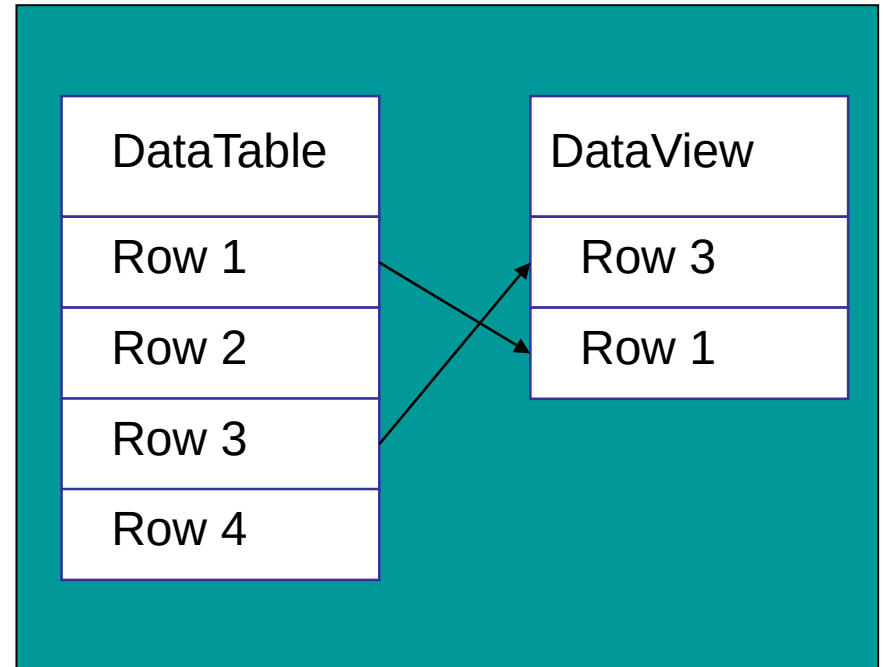
    // The DataTableReader works just like the DataReader.
    while (dtReader.Read())
    {
        for (int i = 0; i < dtReader.FieldCount; i++)
        {
            Console.Write("{0}\t",
                           dtReader.GetValue(i).ToString());
        }

        Console.WriteLine();
    }

    dtReader.Close();
}
```

DataView

- отбирает записи, удовлетворяющие заданному условию;
- отбирает записи, находящиеся в определенном состоянии;
- сортирует записи по значениям указанных столбцов;
- МОЖЕТ ОСНОВЫВАТЬСЯ ТОЛЬКО на одной таблице.



```
coltsOnlyView = new DataView(inventoryTable);  
// Now configure the views using a filter.  
coltsOnlyView.RowFilter = "Make = 'Colt'";  
// Bind to the new grid.  
dataGridColtsView.DataSource = coltsOnlyView;
```


Создание DataView

```
DataTable tbl = new DataTable("Customers");
```

```
// Инициализация таблицы Customers ...
```

```
DataViewRowState dvrs =  
    DataViewRowState.ModifiedOriginal |  
    DataViewRowState.Deleted;
```

```
DataView view = new DataView();  
view.Table = tbl;  
view.RowFilter = "Country = 'USA'";  
view.Sort = "City DESC";  
view.RowStateFilter = dvrs;
```

```
DataView view2 = new DataView(tbl,  
    "Country = 'USA'", "City DESC", dvrs);
```

Загрузка данных из БД и отправка измененных данных

```
string connectionString = "Data Source=localhost;" +  
    "Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);  
  
SqlDataAdapter daCustomers = new SqlDataAdapter("SELECT * FROM Customers", conn);  
SqlDataAdapter daOrders = new SqlDataAdapter("SELECT * FROM Orders", conn);  
SqlDataAdapter da = new SqlDataAdapter(  
    "SELECT OrderID, ProductID, Quantity, UnitPrice FROM [Order Details]", conn);  
  
ds = new DataSet();  
SqlCommandBuilder cb = new SqlCommandBuilder(da);  
  
daCustomers.Fill(ds);  
daOrders.Fill(ds);  
da.Fill(ds);  
  
...  
  
da.Update(tbl);
```

- Fill()
- SelectCommand
- InsertCommand
- UpdateCommand
- DeleteCommand
- Update()

Настройка запросов DataAdapter

```
SqlCommand cmd = new SqlCommand(  
    "UPDATE [Order Details] " +  
    "SET OrderID = ?, ProductID = ?, " +  
    "Quantity = ?, UnitPrice = ? " +  
    "WHERE OrderID = ? AND ProductID = ? AND " +  
    "Quantity = ? AND UnitPrice = ?", conn);
```

```
SqlParameterCollection pc = cmd.Parameters;  
pc.Add("OrderID_New", SqlDbType.Integer, 0, "OrderID");  
pc.Add("ProductID_New", SqlDbType.Integer, 0, "ProductID");  
pc.Add("Quantity_New", SqlDbType.SmallInt, 0, "Quantity");  
pc.Add("UnitPrice_New", SqlDbType.Currency, 0, "UnitPrice");
```

```
SqlParameter param;  
param = pc.Add("OrderID_Orig", SqlDbType.Integer, 0, "OrderID");  
param.SourceVersion = DataRowVersion.Original;  
param = pc.Add("ProductID_Orig", SqlDbType.Integer, 0, "ProductID");  
param.SourceVersion = DataRowVersion.Original;  
param = pc.Add("Quantity_Orig", SqlDbType.SmallInt, 0, "Quantity");  
param.SourceVersion = DataRowVersion.Original;  
param = pc.Add("UnitPrice_Orig", SqlDbType.Currency, 0, "UnitPrice");  
param.SourceVersion = DataRowVersion.Original;
```

```
dataAdapter.UpdateCommand = cmd;
```

Настройка запросов DataAdapter: использование CommandBuilder

```
string = "Data Source=localhost;" +  
    "Initial Catalog=Northwind;Integrated Security=SSPI;"  
SqlConnection conn = new SqlConnection(connectionString);
```

```
SqlDataAdapter da = new SqlDataAdapter(  
    "SELECT OrderID, ProductID, Quantity, UnitPrice FROM [Order Details]",  
    connectionString);
```

```
ds = new DataSet();  
SqlCommandBuilder cb = new SqlCommandBuilder(da);
```

```
da.Fill(ds);  
...  
da.Update(tbl);
```

- ограничения использования CommandBuilder:
 - в запросе выборки участвует единственная таблица;
 - для данной таблицы определен первичный ключ;
 - поля первичного ключа содержатся в запросе выборки и объекте DataTable.

Вопросы к экзамену

- ADO.NET: режимы работы, объектная модель, базовые интерфейсы и базовые классы. Поставщики данных.
- Связный уровень ADO.NET. Объекты связанного уровня и общий сценарий работы.
- Связный уровень ADO.NET. Общий сценарий работы. Пул подключений.
- Связный уровень ADO.NET. Поставщики данных. Создание программного кода, не зависящего от поставщика.
- Связный уровень ADO.NET. Параметризованные запросы.
- Несвязный уровень ADO.NET. Объекты несвязного уровня и общий сценарий работы.
- Несвязный уровень ADO.NET. Настройка запросов для синхронизации с источником данных.

Лабораторная работа №12.

Программирование клиентского приложения доступа к данным

1. Создать пользовательское приложение с использованием технологии ADO.NET для выполнения следующих операций:
 - просмотра содержимого таблиц;
 - вставки, удаления, изменения данных в этих таблицах.
2. Сравнить на практике использование для некоторой таблицы:
 - несвязного уровня ADO.NET;
 - связанного уровня ADO.NET.
3. Для хранения строки подключения к источнику данных использовать конфигурационный файл приложения (app.config).